

Mission 2: Wireshark Analysis Report

Student Name: [Your Name]

Date: April 7, 2026

Course: [Your Course Name]

Introduction

The goal of this mission was to find the "invisible threads" (network connections) between a drone and its controller. By opening the PCAP file in Wireshark, I was able to see exactly who is talking to whom, what language they are using, and what instructions are being sent. Every claim in this report is backed by screenshot evidence from the packet capture.

a. The Interacting Entities

There are two main entities communicating in this file: the **Ground Controller** and the **Drone Autopilot**. Even though they are talking over a local network (simulation), the structure shows exactly how a real drone link works.

1. The Ground Controller (Who is sending commands)

- a. In MAVLink, the controller usually uses **System ID 255** and **Component ID 190** (which stands for Mission Planner or Ground Control Station).
- b. The controller sends out commands like "change mode", "arm", and "takeoff".
- c. **Proof:** See SysID.jpg which shows System id: 255 and Component id: MAV_COMP_ID_MISSIONPLANNER (190).

2. The Drone Autopilot (Who is receiving commands)

- a. The drone uses **System ID 1** and **Component ID 1** (standard ID for the flight controller).
- b. It receives the commands and sends back live status updates (telemetry) and acknowledgment messages.
- c. **Proof:** See SysID.jpg which shows target_system: 1 and target_component: 1.

3. The "Invisible Thread" (Network Connection)

- a. Both devices are communicating over the local IP address `127.0.0.1` (localhost). This means the capture is from a simulated environment, but the network behavior is identical to a real radio link.
- b. **Proof:** See `Conversation_IPV4.jpg` and `Endpoints_IPV4.jpg`, which list `127.0.0.1` as both the source and destination, confirming the direct communication thread.

b. The Interaction Protocol

The controller and drone communicate using a lightweight messaging protocol called **MAVLink (Micro Air Vehicle Link)**. This is the standard "language" for drones.

1. MAVLink Version 2.0

- a. The packets show `Magic value / version: MAVLink 2.0 (0xfd)`. Version 2 is the modern standard and supports longer messages and better security features.
- b. **Proof:** See `mavlink_message_76.jpg` under the MAVLink Protocol header.

2. Transport Layer (How the messages travel)

- a. MAVLink messages are wrapped inside **UDP** and **TCP** packets to travel over the network.
- b. The traffic uses standard MAVLink ports: **14550** and **14445**. Port 14550 is typically used by the ground station to send commands, and 14445 is used by the drone to send telemetry back.
- c. **Proof:** See `Conversation_UDP.jpg` which shows traffic between ports 14550 and 14445. See `Protocol_Hierarchy.jpg` which lists `udp` and `tcp` directly under the `mavlink` protocol tree.

3. Why this matters

- a. Knowing the protocol and ports allows anyone monitoring the network to filter, read, and even predict drone behavior. This is exactly how the "invisible threads" are tapped into.

c. Instructions, Commands, Success/Failure & Adversarial Information

1. Command Sequence & Verification

I tracked a complete flight mission. The controller sent four clear instructions, and the drone confirmed each one.

Step	Command Sent	Screenshot Proof	Result
1	Change flight mode to GUIDED	mode_change.jpg	Accepted
2	ARM the motors	arm_disarm.jpg	Accepted
3	TAKEOFF to 30 meters	takeoff_30m.jpg	Accepted
4	Change mode to LAND	land_mode.jpg	Accepted

- **Success Verification:** After each command, the drone sent back a `COMMAND_ACK` message with `result: MAV_RESULT_ACCEPTED (0)`. This means the drone understood the command and executed it successfully.
- **Proof:** See `ACK_Mode_Change.jpg`, `ACK_Takeoff.jpg`, and `ACK_LAND.jpg`. All show `result: ACCEPTED (0)`.

2. Adversarial Information & Justification

Adversarial information is any data that an attacker could use to disrupt, hijack, or destroy the drone. Based on the hint provided, *any* data becomes adversarial depending on how it is used. Here is what an adversary could learn from this capture:

A. Exact Altitude & Flight Phase

- **Data Found:** `takeoff_alt.jpg` and `Max_alt_30m.jpg` show the drone reached exactly **30.389 meters**. `land_alt.jpg` shows it descending.
- **Why it is adversarial:** An attacker can use altitude data to time their actions. For example, drone jammers or GPS spoofers work best at certain heights. Knowing the drone is at 30m tells an adversary exactly when to activate interference. It also reveals the flight profile, making it easier to predict where the drone will be next.

B. Flight Mode & Control State

- **Data Found:** `mode_change.jpg` and `land_mode.jpg` show the drone switching to GUIDED mode, then LAND mode.
- **Why it is adversarial:** GUIDED mode means the drone is following external coordinates, not flying itself. LAND mode means it is on a fixed, automated descent path. An adversary knowing this can predict the drone's exact trajectory and set up physical or electronic traps along that path.

C. Armed Status & System Health

- **Data Found:** `arm_disarm.jpg` confirms the drone is armed. ACK messages confirm the system is responsive.
- **Why it is adversarial:** "Armed" means the motors are spinning and the drone is active. An adversary now knows the drone is live and dangerous to approach. It also confirms the communication link is stable, so an attacker might try a Denial-of-Service (DoS) attack on the MAVLink ports to sever control.

D. Network Endpoints & Ports

- **Data Found:** `Conversation_IPV4.jpg` and `Endpoints_IPV4.jpg` show the exact IP addresses and ports (14550, 14445) used for control.
- **Why it is adversarial:** If this were a real radio link, an attacker could target these specific ports with flood attacks to crash the ground station software, or inject fake MAVLink packets to hijack the drone. Knowing the "thread" structure makes it easier to cut or take over.

Proof for Section C: All claims are directly supported by the labeled screenshots: `takeoff_alt.jpg`, `Max_alt_30m.jpg`, `land_alt.jpg`, `mode_change.jpg`, `land_mode.jpg`, `arm_disarm.jpg`, `ACK_Takeoff.jpg`, `ACK_LAND.jpg`, `Conversation_IPV4.jpg`, and `Endpoints_IPV4.jpg`.

Conclusion

By analyzing the PCAP file, I successfully identified the two interacting entities (Ground Controller SysID 255 and Drone SysID 1), confirmed they communicate using MAVLink 2.0 over UDP/TCP ports 14550/14445, and reconstructed a full command sequence (Guided → Arm → Takeoff 30m → Land). Every step was verified with `COMMAND_ACK` messages. The

telemetry, mode changes, altitude data, and network endpoints all qualify as adversarial information because they give an attacker the exact timing, location, control state, and network targets needed to disrupt or hijack the mission. All findings are backed by screenshot evidence as required.